Mythos Governance Engine — Security Gate Spec (v5)

Purpose

Deterministic CI/CD governance system that evaluates every code change as a structured ChangeRequest, applies Mythos-class re-pricing, computes blast radius, and enforces barrier-based deployment rules across software surfaces.

Core Principles

- **Barrier > Friction**
  - Only cryptographic, sandbox, or hard-isolation controls are considered security boundaries.
  - Friction (rate limits, heuristics, filters) has no authoritative security value.
- **Blast Radius First**
  - System changes are evaluated based on potential system-wide impact.
  - Critical blast radius changes cannot reach runtime.
- **Mythos-Class Re-price**
  - All advisory severities are re-evaluated assuming an advanced automated adversary capable of systematic exploitation of known and novel vulnerability classes at scale.
  - Advisory labels are not trusted as authoritative.
- **Anchoring Protection**
  - The system detects divergence between advisory severity and re-priced severity. Divergence exceeding one blast-radius level flags the change for manual review regardless of re-price outcome.
- **Surface Isolation**
  - Every change belongs to exactly one Surface defining privilege boundaries.

Surfaces

Surface	Privilege Boundary	Valid Action Types
runtime	Production execution environment	code_change, deploy, config_change, schema_change, auth_change, payment_change
dev-build-attacker-input	Untrusted input processing during build	dependency_update, config_change
dev-build-supply-chain	External dependency ingestion	dependency_update
dev-build-credential-exposure	Build-time credential access or exposure detection surface	*(none —see Rule 1)*
dev-build-isolated	Sandboxed build with no privilege	code_change, config_change

Note: `dev-build-credential-exposure` is a classification surface only. Any change classified to this surface indicates a build-time credential access pattern. No action type is valid here because

credential systems are never directly modified via CI automation (Invariant #1). This surface exists to ensure such changes are explicitly blocked rather than silently misrouted.

Action Types

- code_change
- deploy
- schema_change
- config_change
- dependency_update
- auth_change
- payment_change

Blast Radius Model

Blast radius is determined **deterministically** by the combination of `Surface` and `Action Type` according to the matrix below. Every valid `(Surface, Action Type)` pair maps to exactly one blast radius level.

Surface	code_change	deploy	schema_change	config_change	dependency_update	auth_change	payment_c
runtime	medium	high	high	medium	high	critical	critical
dev-build-attacker-input	—	—	—	medium	high	—	—
dev-build-supply-chain	—	—	—	—	high	—	—
dev-build-isolated	low	—	—	low	—	—	—
dev-build-credential-exposure	*(blocked — Rule 1)*	*(blocked —Rule 1)*	*(blocked — Rule 1)*	*(blocked — Rule 1)*	*(blocked —Rule 1)*	*(blocked — Rule 1)*	*(blocked —1 1)*

Legend: — = invalid action type for that surface (treated as BLOCK). The credential-exposure surface is unconditionally blocked per Rule 1.

Level	Definition	Advisory Severity Mapping (example)
low	UI, isolated logic changes with no system-wide impact	CVSS 0.0–3.9, vendor “low” or “informational”

Table 3 – continued

Level	Definition	Advisory Severity Mapping (example)
medium	Code or config updates affecting runtime behavior, limited scope	CVSS 4.0–6.9, vendor “medium”
high	Schema changes, deploy operations, infrastructure changes, supply-chain ingestion	CVSS 7.0–8.9, vendor “high”
critical	Authentication, payment, or credential system changes	CVSS 9.0–10.0, vendor “critical” or “severe”

Note: The advisory severity mapping is a reference heuristic. The re-price module may use alternative severity taxonomies provided they map cleanly to the 4-level blast radius scale. If a taxonomy cannot be mapped, re-price downgrade is prohibited.

Barrier Systems (Hard Security Truths)

Barrier systems are required controls that must be active for specific outcomes to be valid.

Barrier	Required For	Active State Definition	Description
cryptographicAuth	ALLOW_RUNTIME, ALLOW_STAGING	Identity verification system reports healthy and all required signatures are valid	All runtime-bound changes must pass cryptographic identity verification
credentialVaultIsolation	ALLOW_RUNTIME, ALLOW_STAGING	Vault reports sealed status; no credential material accessible from build environment	Credential vault must be cryptographically isolated from build environment
paymentIsolation	ALLOW_RUNTIME, ALLOW_STAGING	Payment network segmentation confirmed; no payment system interfaces reachable from build/staging	Payment systems must be isolated from build and staging environments
sandboxExecution	ALLOW_SANDBOX	Sandbox reports enforced boundary; process cannot escape via namespace, seccomp, or hardware isolation	Sandbox boundary must be enforced with hardware or kernel-level isolation
productionDeploymentGate	ALLOW_RUNTIME	Deployment gate reports passable; all required approvals and checks satisfied	Explicit production deployment gate must be active and passable

Note: A barrier is "active" when its health check reports a positive status at the time of evaluation. Barrier health checks are out-of-band from the governance engine and must be provided by the underlying infrastructure.

Friction Systems (Non-authoritative)

- rate limiting
- spam filtering
- heuristic moderation
- keyword detection

Friction cannot block or authorize critical decisions. Friction systems may supplement barrier systems for operational efficiency but never substitute for them.

Decision Outcomes

Outcome	Description
ALLOW_RUNTIME	Approved for production deployment. Requires all runtime barriers active.
ALLOW_STAGING	Approved for staging environment only. Requires cryptographicAuth, credentialVaultIsolation, and paymentIsolation barriers active. Promotion to runtime requires a new ChangeRequest evaluation.
ALLOW_SANDBOX	Approved for sandboxed execution only. Requires sandboxExecution barrier active.
DEFER	Suspended pending manual review. Must include valid deferral structure per Defer Policy.
BLOCK	Unconditionally denied. Cannot be deferred, overridden, or re-priced.

Evaluation Precedence Hierarchy

Evaluation proceeds in strict order. The first applicable rule determines the outcome ceiling. Subsequent steps may only restrict further or relax within the ceiling.

1. **Surface Rule** (Rule 1) — unconditional hard block
2. **Blast Radius Rule** (Rules 2–5) — determines environment ceiling
3. **Barrier Verification** — confirms required barriers are active; if any required barrier is inactive, outcome is demoted to the next lower permissible outcome or BLOCK
4. **Mythos-Class Re-price Modifier** — may downgrade (never upgrade) the blast-radius-derived outcome if re-priced severity is lower AND anchoring protection does not flag divergence

Clarification: Blast radius is the primary enforcement dimension. Re-price logic can only relax restrictions when the re-priced severity is demonstrably lower than the blast-radius-implied severity

AND no anchoring divergence is detected. Re-price can never escalate permissions beyond what blast radius allows. Re-price is a modifier, not a competing rule.

Evaluation Rules

1. Credential Surface Hard Block

Any change targeting the `dev-build-credential-exposure` surface is immediately `BLOCK`, regardless of action type or blast radius. No action type is valid on this surface.

> **Rationale:** Credential systems are never directly modified via CI automation (Invariant #1).

> **Interaction with other rules:** Rules 2–5 do not apply to this surface. A BLOCK per Rule 1 cannot be deferred, re-priced, or overridden.

2. Critical Blast Radius Rule

All critical blast radius changes must be `DEFER`.

> **Rationale:** Critical blast radius cannot reach runtime (Invariant #2). Defer is mandatory; no re-price relaxation applies.

3. High Blast Radius Rule

High blast radius changes are restricted as follows:

- On `runtime` surface: restricted to `ALLOW_STAGING`
- On `dev-build-supply-chain` surface: restricted to `ALLOW_SANDBOX`
- On `dev-build-attacker-input` surface: restricted to `ALLOW_SANDBOX`
- On `dev-build-isolated` surface: restricted to `ALLOW_SANDBOX`
- On `dev-build-credential-exposure` surface: `BLOCK` (Rule 1)

> **Rationale:** High blast radius changes are never allowed directly into runtime, regardless of surface. The most permissive outcome for high blast radius is staging (runtime surface) or sandbox (non-runtime surfaces).

4. Medium Blast Radius Rule

Medium blast radius changes are limited to `ALLOW_SANDBOX` on all surfaces **except** `dev-build-credential-exposure` (blocked per Rule 1).

5. Low Blast Radius Rule

Low blast radius changes may proceed to `ALLOW_RUNTIME` on all surfaces **except** `dev-build-credential-exposure` (blocked per Rule 1).

Compound ChangeRequest Handling

A ChangeRequest may contain multiple `(Surface, Action Type)` pairs. The engine evaluates each pair independently and produces a per-pair outcome. The final ChangeRequest outcome is the **most restrictive** outcome across all pairs, determined by the ordering:

`BLOCK` > `DEFER` > `ALLOW_SANDBOX` > `ALLOW_STAGING` > `ALLOW_RUNTIME`

If any pair maps to an invalid `(Surface, Action Type)` combination, the entire ChangeRequest is `BLOCK`.

Mythos-Class Re-price Procedure

Re-price Input

The re-price module receives:

- Advisory severity (e.g., CVSS score, vendor severity label)
- Blast radius level derived from `(Surface, Action Type)` matrix
- Change metadata (affected components, exposure surface area, exploit prerequisites)

> **Note:** If no advisory severity is available for a change, re-price downgrade is prohibited. The change retains its matrix-derived outcome. Anchoring divergence cannot be computed without advisory severity, so the conservative path (no downgrade) is enforced.

Re-price Output

The re-price module produces a **re-priced blast radius level** (low, medium, high, or critical) based on adversarial analysis.

Downgrade Conditions

Re-price may downgrade the outcome **only if**:

1. The re-priced blast radius level is strictly lower than the matrix-derived blast radius level.
2. The anchoring divergence check does not flag the change.
3. The outcome after downgrade does not require barriers that are currently inactive (barrier requirements are hard constraints that re-price cannot override).

Anchoring Divergence

Anchoring divergence is detected when the absolute difference between advisory severity (mapped to the 4-level blast radius scale) and re-priced blast radius level exceeds one level. When divergence is detected, the change is flagged for manual review and re-price downgrade is **prohibited**.

> **Example:** If advisory severity maps to `critical` but re-price produces `medium`, divergence is detected (gap of 2 levels). The change retains its matrix-derived outcome and is flagged.

Barrier Verification Procedure

After blast radius rules determine the outcome ceiling, the engine verifies that all barrier systems required for that outcome are active.

Outcome	Required Barriers	Failure Mode
ALLOW_RUNTIME	cryptographicAuth, credentialVaultIsolation, paymentIsolation, productionDeploymentGate	If any barrier inactive → BLOCK
ALLOW_STAGING	cryptographicAuth, credentialVaultIsolation, paymentIsolation	If any barrier inactive → ALLOW_SANDBOX (if sandboxExecution active) or BLOCK
ALLOW_SANDBOX	sandboxExecution	If inactive → BLOCK
DEFER	None	Deferral structure must be valid per Defer Policy
BLOCK	None	Immediate termination

Outcome Lifecycle & Promotion

Current Outcome	Promotion Path	Requirements
ALLOW_SANDBOX	→ ALLOW_STAGING	New ChangeRequest evaluation with additional review
ALLOW_STAGING	→ ALLOW_RUNTIME	New ChangeRequest evaluation; all runtime barriers (cryptographicAuth, credentialVaultIsolation, paymentIsolation, productionDeploymentGate) must be active
DEFER	→ Any allowed outcome	Owner resolves trigger condition; new ChangeRequest evaluation required
BLOCK	None	Permanent; cannot be promoted

Promotion requires a **new ChangeRequest** evaluated against the full rule set. Outcomes do not auto-promote based on time or environment state.

Defer Policy

A valid defer must include all of the following:

- **owner** — accountable party (individual or team identifier)
- **trigger** — concrete event condition, explicit date, or measurable state change for re-evaluation. Triggers must be specific and verifiable; vague triggers (e.g., “when reviewed”) are rejected as invalid.

- **interim control** — compensating control active during deferral

Invalid defer = `BLOCK` plus security alert. The system must not enter an unmanaged state. An invalid defer structure is treated as a failed evaluation and the ChangeRequest is blocked with an alert escalated to the security operations team.

A defer cannot be applied to a `BLOCK` outcome. Only blast-radius-derived outcomes (`ALLOW_RUNTIME`, `ALLOW_STAGING`, `ALLOW_SANDBOX`) may be deferred to enable manual review.

Invariants

- Credential systems are never directly modified via CI automation.
- Critical blast radius cannot reach runtime.
- Advisory severity never overrides re-price logic (re-price may only relax, never escalate).
- All deferrals must be structurally valid.
- Friction systems are never used as security barriers.
- Outcome promotion requires a new ChangeRequest evaluation.
- The most restrictive per-pair outcome governs the entire ChangeRequest.

System Definition

The Mythos Governance Engine is a deterministic CI enforcement system that evaluates all changes across software surfaces, computes blast radius via a deterministic `(Surface, Action Type)` matrix, applies Mythos-class re-pricing with anchoring protection, enforces strict barrier-based deployment control, and ensures no unsafe production transitions occur.